How to use CMake And Clang-Tidy:

By Shachar Tsrafati

Note: This guide will show in Windows but the step should be the same in mac

Before I will explain how to use CMake and TIdy, I will explain what is CMake and TIdy

So what is CMake ?

CMake is **not a compiler**. It is a **build system generator**.

You may have already written complex **Makefiles** in the System Programming 1 course, or manually configured projects in IDEs such as **Visual Studio** or **CLion**.

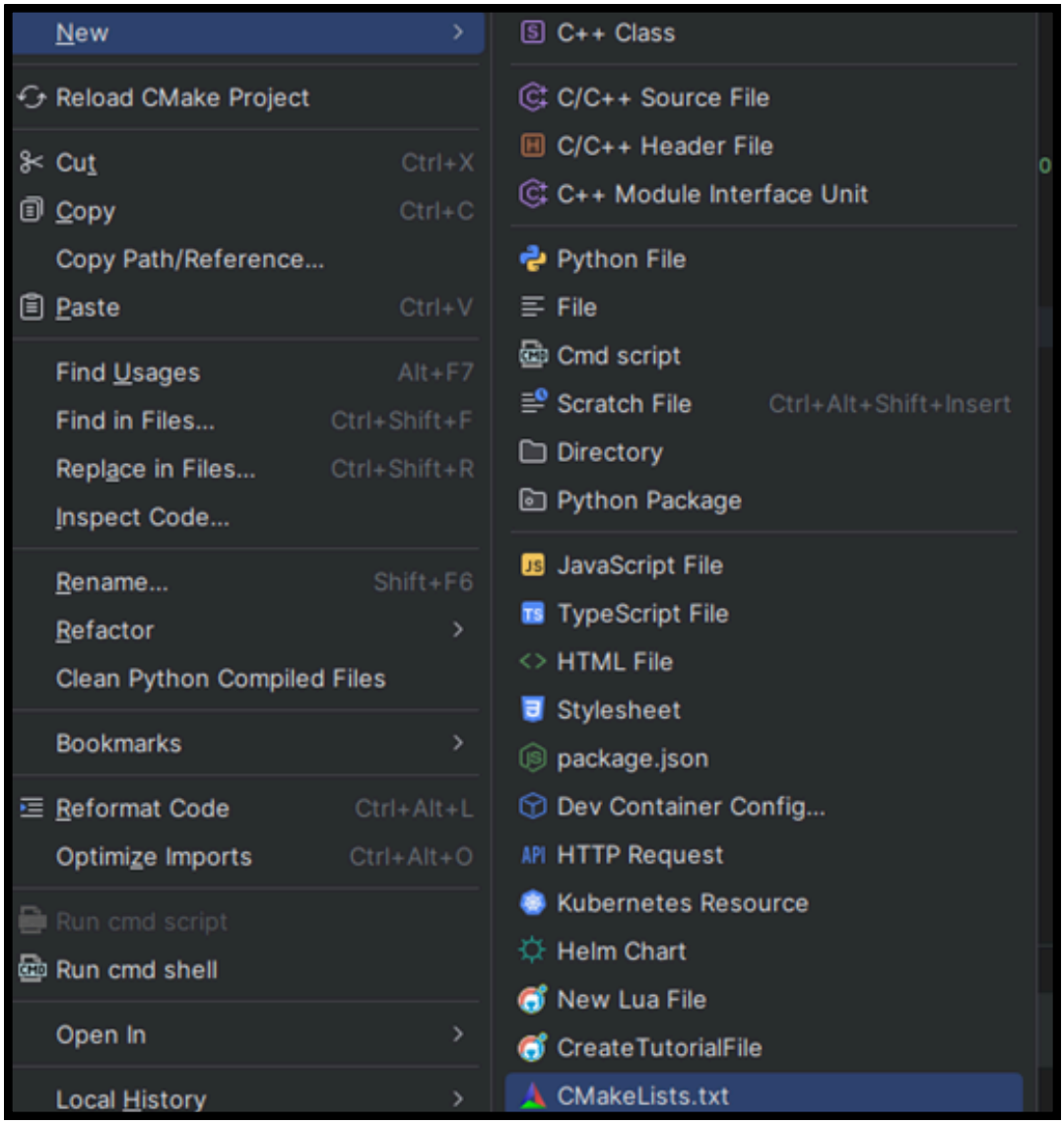
Instead of writing build files manually, CMake allows you to define the project configuration in a simple file. From this configuration, CMake automatically generates the required build system (for example **Makefiles**, **Ninja**, or **Visual Studio project files**).

In the root directory of your project, you need to create a configuration file called:

CMakeLists.txt

It is **very important that the filename is exactly this**, otherwise CMake will not recognize it.

Some IDEs such as **CLion** can generate this file automatically, and in many development environments you can create it by **right-clicking the project folder and selecting "New → CMakeLists.txt"**.



Why CMake?

• **Cross-platform support:**
CMake is a cross-platform build system. You define the project configuration once, and CMake can generate build files for different operating systems such as **Linux, Windows, and macOS**.

• **Library management:**
CMake makes it easier to work with external libraries. You can specify that your project depends on a certain library, and **CMake** will help locate it on the system and link it to your project.

• **Automation:**
When a new source file is added to the project, you usually only need to add it to the source list in **CMakeLists.txt**. CMake then takes care of generating the necessary build rules automatically.

Why Clang-Tidy?

Clang-Tidy is a static analysis tool for C and C++ code. It analyzes the source code without running the program and helps detect potential problems.
We use Clang-Tidy for several reasons:

• **Find bugs early:**
Clang-Tidy can detect common programming mistakes, undefined behavior, and potential runtime errors before the program is executed.

• **Improve code quality:**
It checks the code against best practices and modern C++ guidelines, helping developers write safer and cleaner code.

• **Enforce coding standards:**
Teams can use Clang-Tidy to ensure that all code follows the same style and rules.

• **Automatic suggestions and fixes:**
Some issues detected by Clang-Tidy can be automatically fixed or improved by the tool.

By using Clang-Tidy regularly during development, programmers can catch issues earlier and maintain higher code quality.

How to use CMake ?

Very simple you need to know a few commands. Lets assume your project looks like this

```
MyProject/
├── CMakeLists.txt   # main configuration file
├── main.cpp         # application source code
├── my_lib.cpp       # library implementation
└── my_lib.hpp       # library header file
```

And these are our files:

my_lib.hpp:

```
#ifndef MY_LIB_HPP
#define MY_LIB_HPP
        void print_hello();
#endif
```

my_lib.cpp:

```
#include "my_lib.hpp"
#include <iostream>
void print_hello() {
   std::cout << "Hello from our custom library!" << std::endl;
}
```

So if we need to make a makefile like old time i will look like this:

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17
all: my_app
my_app: main.cpp my_lib.cpp
   $(CXX) $(CXXFLAGS) main.cpp my_lib.cpp -o my_app
clean:
   rm -f my_app
```

Now look how it is so simple to use CMake instead
Go to your CMakeLists.txt, and type these lines:

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(MyProject)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_CLANG_TIDY
   clang-tidy;
   -checks=cppcoreguidelines-*;
)
add_executable(my_app
main.cpp
my_lib.cpp
)
```

Just like that we complied files in 10 simple lines WOW

How to write CMakeLists.txt

First line is the Cmake minimum version required we will use 3.10 because most of old programs support that version. (We can use higher version but the higher we use, less programs will support).

cmake_minimum_required(VERSION 3.10)

Then we need to say the name of the project just name it whatever you want.

project(MyProjectName)

Then we **CAN add (not must)** define the C++ Version we want:

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CMAKE_CXX_STANDARD 17):
tells CMake to compile the project using the **C++17** standard.
If we can set to 23 or 26 but we will define 17 as Version.
set(CMAKE_CXX_STANDARD_REQUIRED ON):
tells CMake that **C++17 is required**, so it should not use an older C++ version instead.

After that we will add the Clang-Tidy

```
set(CMAKE_CXX_CLANG_TIDY
   clang-tidy;
   -checks=cppcoreguidelines-*;
)
```

CMAKE_CXX_CLANG_TIDY tells CMake to run **Clang-Tidy** on the C++ source files during the build process.
In this example, we use clang-tidy with the checks group **cppcoreguidelines-***, which enables checks based on the C++ Core Guidelines.
There are many other ways to configure Clang-Tidy, and many additional options and check groups are **available in the official documentation**. However, in this course we compile relatively simple programs, so this line is enough and is a good choice for our needs.

Finally, we need to add our C++ source files:

```
add_executable(my_app
main.cpp
my_lib.cpp
)
```

add_executable() tells CMake to create an executable program.

- The first argument, my_app, is the name of the output program.
- The following arguments are the source files used to build it.
- One of these files must contain the main() function.
- You can add as many additional .cpp files as your project needs.

How I Compiled multiple programs ?

It is so simple just add another "**add_executable**" like this

CMakeList.txt

```
cmake_minimum_required(VERSION 3.10)
project(AutoProject CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_CLANG_TIDY
 clang-tidy;
 -checks=cppcoreguidelines-*
)
add_executable(program_one program_one.cpp)
add_executable(program_two program_two.cpp)
```

We finished to use **CMakeLists.txt** 😄

Now we know how to use **CMakeLists.txt**

How to call CMake ?

Note: This method on Windows as far i know these should work on Mac too and Linux
I'm attaching a YouTube guide in case I say something that isn't exactly correct
I recommend checking what I'm saying for those who use Mac and Linux

setup Mac OS for C++, CMake development(build+debug) using Visual Studio Code

Before I start

CMake choose the default generator for your system it does not mean you need to choose that, I will show you 2 ways
1) For default generator (most of you all use it) any OS (Each OS has it own default)
2) Ninja generator (Sometimes you may need it), Yes you need to download it from the internet and install it. Cross platform.

Why we want to use not the default generator ?

The Problem with "Default"

When you run CMake without specifying a generator, it picks the most common tool for your current OS:
the "Default" generator usually ties you to the specific operating system you are currently using, **which can cause headaches** when you want your code to run everywhere.

Operating System	Default Generator	Native Compiler
Windows	**Visual Studio**	**MSVC** (Microsoft Visual C++)
Linux	**Unix Makefiles**	**GCC** (GNU Compiler Collection)
macOS	**Xcode / Unix Makefiles**	**Apple Clang** (LLVM)

Why Use Ninja (or another specific tool) instead?

1. Consistent Behavior Using a specific toolset like GCC or Clang across all systems ensures your code behaves identically regardless of the host OS. This prevents "it works on my machine" bugs caused by different compilers interpreting the C++ standard differently.

2. Avoiding OS "Lock-in" Relying on a universal toolset prevents you from accidentally using proprietary, OS-specific features (like Windows-only or Mac-specific headers). It forces you to write truly portable code that can move between any system without breaking.

3. Matching the Production Environment Since most servers and Docker containers run Linux, using a matching compiler locally ensures your development environment mimics production. This stops "deployment day" errors where code works on your personal computer but fails once it is uploaded.

4. Tool & Linter Compatibility Professional tools like clang-tidy were built for the LLVM/GCC ecosystem and work most reliably when paired with those specific compilers. Using a cross-platform generator ensures these code-cleaning tools provide the same warnings to everyone on the team.

How to install Ninja generator

First go to this link: https://github.com/ninja-build/ninja/releases

- Windows: ninja-win.zip

Download the latest version
For Windows:

- Download the file
- Go to your C:\Program Files
- Make a folder called "**Ninja**" (so we know it is ninja generator)
- **Add the exe file** to that folder
- Go to **edit environment variables**
- Go to Path and add: C:\Program Files\Ninja
- Open new Terminal and type: **ninja --version** if you see **1.13.2** you done

For macOS
The best way to install Ninja on a Mac is using a package manager:

- **Using Homebrew:** Open your terminal and run: **brew install ninja**
- **Using MacPorts:** Run: **sudo port install ninja**

After installing, open a **new** terminal window and type: **ninja --version**

For Linux:
Most Linux distributions include Ninja in their official repositories:

- **Ubuntu / Debian / Mint:** Run: **sudo apt install ninja-build**

After installing, open a **new** terminal window and type: **ninja --version**

How to call Cmake ?

- Go to your project folder
- Open terminal in that folder (or set path to that folder)

Default generator:
cmake -S . -B build
cmake --build build

Finding the executable program:
Windows:
Inside the build folder there will be a **Debug folder**, you can run using this command from the project folder

.\build\Debug\OurProgram.exe

For Linux / macOS:
Most often:

./build/OurProgram

Ninja:
cmake -S . -B build -G "Ninja"
cmake --build build

Finding the executable program:
Windows:
Inside build folder there will be the executable program (there are no Debug folder)

.\build\OurProgram.exe

For Linux / macOS:
Most often:

./build/OurProgram

Note about “Build” folder

We do **not have to** name the folder **build**.
The name is arbitrary, and CMake will use whatever folder we specify.
For example, instead of:

cmake -S . -B build
cmake --build build

we could use any other name:

cmake -S . -B bananananana
cmake --build bananananana

This would still work the same way. CMake would simply place all generated build files and compiled output inside the folder bananananana. However, using the name **build** is a common convention because it clearly indicates that the folder contains the generated build files and compiled artifacts.

Configuring Static Analysis (Clang-Tidy)

Clang-Tidy is a cross-platform "linter" that analyzes your code for logic errors and style issues without running it. By using specific CMake variables, we ensure this check works the same way whether you are on Windows, Linux, or a Mac

The CMake Configuration
Copy this block into your CMakeLists.txt. It uses universal variables to find the correct files on any OS:

```
# 1. Locate the clang-tidy executable on the current system
find_program(CLANG_TIDY_EXE clang-tidy)

if(CLANG_TIDY_EXE)
  # 2. Define strict rules for all platforms
  set(TIDY_FLAGS
      -checks=bugprone-*,clang-analyzer-*,cppcoreguidelines-*,performance-*,portability-*,-portability-avoid-pragma-once,readability-*
      --warnings-as-errors=*
  )

  # 3. Create a universal shortcut to check only your implementation
  add_custom_target(tidy
      COMMAND ${CLANG_TIDY_EXE} ${CMAKE_SOURCE_DIR}/Student.cpp ${TIDY_FLAGS} -- -std=c++23
  )
endif()
```

Terminal Commands
Run this from your project root folder. CMake handles the OS-specific file extensions (like .exe) automatically:

- Initialize (Run once): cmake -S . -B build

- Run Tidy: cmake --build build -t tidy

Professional Project Hygiene

When using strict compiler flags like -Werror and automated grading tools, you must ensure that your environment is "clean." Two critical steps are managing external libraries and ignoring build "noise."

1. Handling External Libraries (The SYSTEM Flag)
You are using strict flags such as -Werror (treat warnings as errors) and --warnings-as-errors=* for Clang-Tidy.

- The Problem: External libraries like doctest.h might have small warnings in their code. Because of your strict settings, the compiler will fail your project even if your own code is perfect.
- The Solution: Tell CMake to treat these files as SYSTEM headers. This instructs the compiler and Clang-Tidy to ignore warnings found inside these specific folders.

Update your target_include_directories:

```
target_include_directories(test_runner SYSTEM PRIVATE
${CMAKE_CURRENT_SOURCE_DIR}/include)
```

2. Clean Submissions with .gitignore
When you run cmake -S . -B build, CMake generates hundreds of temporary files (like .obj or .ninja files) inside the build/ folder.

- The Rule: You should never upload the build/ folder to GitHub or submit it with your assignment. These files are huge and only work on your specific computer.
- The Solution: Create a file named .gitignore in your project's root directory (next to CMakeLists.txt).

Add these lines to your .gitignore:

```
# Ignore all build artifacts
build/
.cache/
bin/
obj/

# Ignore IDE-specific folders
.vscode/
.idea/
cmake-build-debug/
```

Automated Testing with CTest

Testing ensures your logic is correct. We use CTest, which is CMake's built-in tool for managing tests across different operating systems.

The CMake Configuration
This setup separates your main application from your test logic.

```
# 1. Enable the testing engine for the current OS
enable_testing()

# 2. Create the test executable (CMake adds .exe on Windows automatically)
add_executable(test_runner test.cpp Student.cpp)

# 3. Register the test so CTest can track it
add_test(NAME assignment_tests COMMAND test_runner)

# 4. Create a "Grade" shortcut that builds and tests in one go
if(CLANG_TIDY_EXE)
  add_custom_target(grade
      COMMAND ${CMAKE_CTEST_COMMAND} --output-on-failure
      COMMAND ${CLANG_TIDY_EXE} ${CMAKE_SOURCE_DIR}/Student.cpp
${TIDY_FLAGS} -- -std=c++23
      DEPENDS test_runner
  )
endif()
```

Terminal Commands
These commands work identically on Windows, macOS, and Linux:

To Run Tests Only:

```
ctest --test-dir build --output-on-failure
```

To Run the Full Grade (Build + Test + Tidy):

- **cmake --build build -t grade**

How to debug with cmake

Windows

Step 1: Ensure WSL is Ready
Before anything else, you need a working Linux environment on your Windows machine.

1. Open PowerShell or CMD as Administrator.
2. Run the following command: **wsl --install .** If it's already installed, it will tell you. If not, follow the prompts and restart your computer if asked.
3. Set up your Linux username and password when the terminal pops up.

Step 2: Install Valgrind and Build Tools in WSL
Now we need to install the actual "detective" (Valgrind) and the Linux version of the compiler inside your WSL.

1. In your terminal, run: **wsl sudo apt update**
2. Install the essential tools: **wsl sudo apt install build-essential cmake valgrind -y**

Step 3: Configure your CMakeLists.txt
To make things "generic" and easy to run, add this block to the end of your CMakeLists.txt. This creates a custom "target" that runs Valgrind specifically for your project.

```
cmake_minimum_required(VERSION 3.10)
project(PizzaTime)

set(CMAKE_CXX_STANDARD 17)

# LIST YOUR FILES MANUALLY HERE (No GLOB)
add_executable(
${PROJECT_NAME}
${Your CPP files}
)

# --- Valgrind Shortcut Configuration ---
find_program(VALGRIND valgrind)

if(VALGRIND)
  add_custom_target(memcheck
    COMMAND ${VALGRIND} --leak-check=full --show-leak-kinds=all
./${PROJECT_NAME}
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
    COMMENT "Running Valgrind memory analysis..."
    DEPENDS ${PROJECT_NAME}
  )
endif()
```

Step 4: Run the Commands via Windows CMD
To avoid the path conflicts we saw earlier, we will create a dedicated Linux build folder (**build_linux**)

A. Initialize the Linux Build
This creates the necessary metadata for Linux. You only need to do this once
(or if you add a new file to the list).
wsl cmake -S . -B build_linux

B. Run the Memory Check
Run this command every time you want to compile and check for leaks. It handles everything in one go:
wsl cmake --build build_linux -t memcheck

How to Read the Results
Valgrind will output a lot of text. Focus on the very bottom:
- HEAP SUMMARY: All heap blocks were freed -- no leaks are possible
  - Meaning: Perfect! Your Destructor is working correctly.
- definitely lost: X bytes in Y blocks
  - Meaning: You have a leak. Usually, this means you missed a delete or a delete[].

MacOs

Step 1: Install Homebrew (The Mac Package Manager)
On Windows, we used apt. On Mac, we use Homebrew. If you don't have it, open your Terminal and run:

/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

Step 2: Install the Tools
You need a compiler and the memory checking tools.
1. Install Command Line Tools: **xcode-select --install**
2. Install Valgrind (Intel Macs only): If you have an older Intel Mac, run: **brew install valgrind**

Note: If you have an M1/M2/M3 Mac, Valgrind isn't fully supported yet. We will use the built-in leaks tool instead.

Step 3: Configure your CMakeLists.txt (Mac Native)
This version assumes you are on a Mac and want to use the built-in leaks tool (which works on all Macs, including M1/M2/M3) without any conditional checks.

```
cmake_minimum_required(VERSION 3.10)
project(PizzaTime)

set(CMAKE_CXX_STANDARD 17)

# LIST YOUR FILES MANUALLY HERE
add_executable(${PROJECT_NAME}
   main.cpp
   PizzaBox.cpp
)

# --- Mac Memory Check Shortcut ---
# This uses the native Apple 'leaks' tool directly
add_custom_target(memcheck
   COMMAND leaks --atExit -- ./${PROJECT_NAME}
   WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
   COMMENT "Running Apple 'leaks' analysis..."
   DEPENDS ${PROJECT_NAME}
)
```

Step 4: Run the Commands via Mac Terminal

On a Mac, you do not use the wsl prefix. You run everything directly in the Terminal app.

A. Initialize the Build

This creates the build folder and prepares the project. You only need to do this once (or if you add a new .cpp file to the list).

```
cmake -S . -B build
```

B. Run the Memory Check

Run this command every time you want to compile and check for leaks. It will build your code and then immediately trigger the leaks tool.

```
cmake --build build -t memcheck
```

How to know if your code does not have any memory leak

1. For Windows (Using WSL)

You are using a "Linux bridge," so you must use the wsl command.

A. Initialize (Once):

```
wsl cmake -S . -B build_linux
```

B. Run the Check (Every time):

```
wsl cmake --build build_linux -t memcheck
```

2. For Mac (Native Terminal)

Since Mac is Unix-based, you run commands directly.

A. Initialize (Once):

cmake -S . -B build

B. Run the Check (Every time):

cmake --build build -t memcheck

```
>wsl cmake -S . -B build_linux
-- The C compiler identification is GNU 13.3.0
-- The CXX compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
```

```
>wsl cmake --build build_linux -t memcheck
[ 33%] Building CXX object CMakeFiles/PizzaTime.dir/main.cpp.o
[ 66%] Building CXX object CMakeFiles/PizzaTime.dir/PizzaBox.cpp.o
[100%] Linking CXX executable PizzaTime
[100%] Built target PizzaTime
gmake[3]: Warning: File 'PizzaTime' has modification time 0.25 s in the future
==1953== Memcheck, a memory error detector
==1953== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1953== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1953== Command: ./PizzaTime
==1953==
--- Program Started ---
```

```
==1762== LEAK SUMMARY:
==1762==    definitely lost: 96 bytes in 3 blocks
==1762==    indirectly lost: 0 bytes in 0 blocks
==1762==      possibly lost: 0 bytes in 0 blocks
==1762==    still reachable: 0 bytes in 0 blocks
==1762==         suppressed: 0 bytes in 0 blocks
==1762==
```

If everything is good so you should get this output (output without any leaks)

```
--- Program Finished Successfully ---
==1824==
==1824== HEAP SUMMARY:
==1824==     in use at exit: 0 bytes in 0 blocks
==1824==   total heap usage: 6 allocs, 6 frees, 74,872 bytes allocated
==1824==
==1824== All heap blocks were freed -- no leaks are possible
==1824==
==1824== For lists of detected and suppressed errors, rerun with: -s
==1824== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[100%] Built target memcheck
```

Helpful Resources & Troubleshooting

If you encounter installation errors, command failures, or bugs you don't understand, remember that setup is often the hardest part of programming. Use these resources to get unstuck:

1. Where to Get Help

- **AI Assistants**: Best for explaining specific error messages in plain English or suggesting quick code fixes.
- **Stack Overflow**: The go-to site for searching exact error codes. If you have an error, someone else has definitely solved it there.
- **GeeksforGeeks**: Great for learning the theory behind concepts like "CMake basics" or "C++ Memory Management."

2. When to Use Them

- **Installation Issues**: Use these if wsl --install or brew install fails.
- **Command Errors**: Use these if you see "Command not found" or "Permission denied."
- **Logic Bugs**: Use these if the code runs but the memory check shows a leak.
-

Pro Tip: Always copy and paste the exact error message into your search.
These tools are made for you to use them don't hesitate!